

Programozási technológia

1. Beadandó

Benedek Marcell

Y1FBMY

Feladat leírása

Szimuláljuk az alábbi egyszerűsített Capitaly társasjátékot! Adott néhány eltérő stratégiájú játékos és egy körpálya, amelyen különféle mezők sorakoznak egymás után. A pályát körbe-körbe újra és újra bejárják a játékosok úgy, hogy egy kockával dobva mindig annyit lépnek, amennyit a kocka mutat. A mezők három félék lehetnek: ingatlanok, szolgáltatások és szerencse mezők. Az ingatlant meg lehet vásárolni 1000 Petákért, majd újra rálépve házat is lehet rá építeni 4000 Petákért. Ha ezután más játékos erre a mezőre lép, akkor a mező tulajdonosának fizet: ha még nincs rajta ház, akkor 500 Petákot, ha van rajta ház, akkor 2000 Petákot. A szolgáltatás mezőre lépve a banknak kell befizetni a mező paramétereként megadott összeget. A szerencse mezőre lépve a mező paramétereként megadott összegű pénzt kap a játékos. Háromféle stratégiájú játékos vesz részt a játékban.

Kezdetben mindenki kap egy induló tőkét (10000 Peták), majd a „mohó” játékos ha egy még gazdátlan ingatlan mezőjére lépett, vagy övé az ingatlan, de még nincs rajta ház, továbbá van elég tőkéje, akkor vásárol. Az „óvatos” játékos egy körben csak a tőkéjének a felét vásárolja el, a „taktikus” játékos minden második vásárlási lehetőséget kihagyja. Ha egy játékosnak fizetnie kell, de nincs elegendő pénze, akkor kiesik a játékból, házai elvesznek, ingatlanjai megvásárolhatókká válnak.

A játék paramétereit egy szövegfájlból olvassuk be. Ez megadja a pálya hosszát, majd a pálya egyes mezőit. Minden mezőről megadjuk annak típusát, illetve ha szolgáltatás vagy szerencse mező, akkor annak pénzdíját. Ezt követően a fájl megmutatja a játékosok számát, majd sorban minden játékos nevét és stratégiáját. A tesztelhetőséghez fel kell készíteni a megoldó programot olyan szövegfájl feldolgozására is, amely előre rögzített módon tartalmazza a kockadobások eredményét. Írjuk ki, melyik játékos esik ki másodszorra a játékból!

Feladat értelmezése

A feladat a Capital társasjáték szimulálására szólít fel. Beolvasunk játékosokat, mezőket, majd dobásokat, ami alapján létrehozuk a játékosokat és mezőket. A main-ben olvasunk be, és a játékosok és mezők példányai a table objektumban vannak tárolva.

A játékosok különböző stílusait örökléssel oldjuk meg, a mezők típusaival szintúgy. A játékosok rendelkeznek névvel, tőkével, aktív státusszal és pozícióval a táblán. A taktikus játékos továbbá körszámlálóval. A mezők rendelkeznek egyedi (sor/azonosító) számmal állapottal, ami egy enum a különböző birtoklási szerkezetekre, illetve tulaj adattaggal.

A játék során ha játékos interaktál valamivel, az aktív státusz vizsgálva van, ennél fogva, ha kiesik, az csak negatív lesz, és ki lesz hagyva. A kiesett játékosokat továbbá eltároljuk, így kapjuk majd meg a második kiesettét.

Program tervezete

A feladatot objektumorientáltan oldom meg. Bekérés egy try-catch blokkal működik, rossz file esetén visszadob. Jó fájl esetén a formátumnak megfelelően olvassa be az adatokat. A formátum, első sorban „predetermined”, a dobások szimulálására, benne hagytam ezt a sort, mert így nyílt bővítésre, de körülbelül 4 kódsor kivenni. Majd beolvassa a játékosok, mezők, illetve dobássorozatok számát. Egy dobássorozatban mindig annyi dobásnak kell legyen, mint amennyi játékos van. Ha egy játékos időközben kiesik, csak átugorja, de a dobást magát felhasználja.

Majd beolvassa a játékosok nevét, kezdőtőkéjét és stratégiáját, ami „moho”, „ovatos”, vagy „taktikus” lehet.

Hasonlóan a mezőkkel, ami „property”, „luckytile”, vagy „unluckytile”. Lucky, illetve unlucky tile esetén beolvas utána egy számot, ami a növelendő/levonandó mennyiség rálépés esetén.

Minden játékos típus egy játékos ősosztályból van származtatva, és kiegészítve a maguk vásárlási mintáival. A vásárlások visitor programozási mintával vannak megoldva, ezért a tryBuy metódus mindre működni fog.

Hasonlóan a mezők is egy mező ősosztályból vannak származtatva, és rálépés esetén különböző hatásokat váltanak ki. Property esetén a tryBuy-ban dől el, hogy sikeres-e a vásárlás. Bekérésnél megszámolja a létező mezők számát. (0-ról inkrementálva természetesen) Mindig egy új számot ad egy új mezőnek. Ez funkcionál részben egy id-nek is, illetve a helyi navigációt is ez biztosítja.

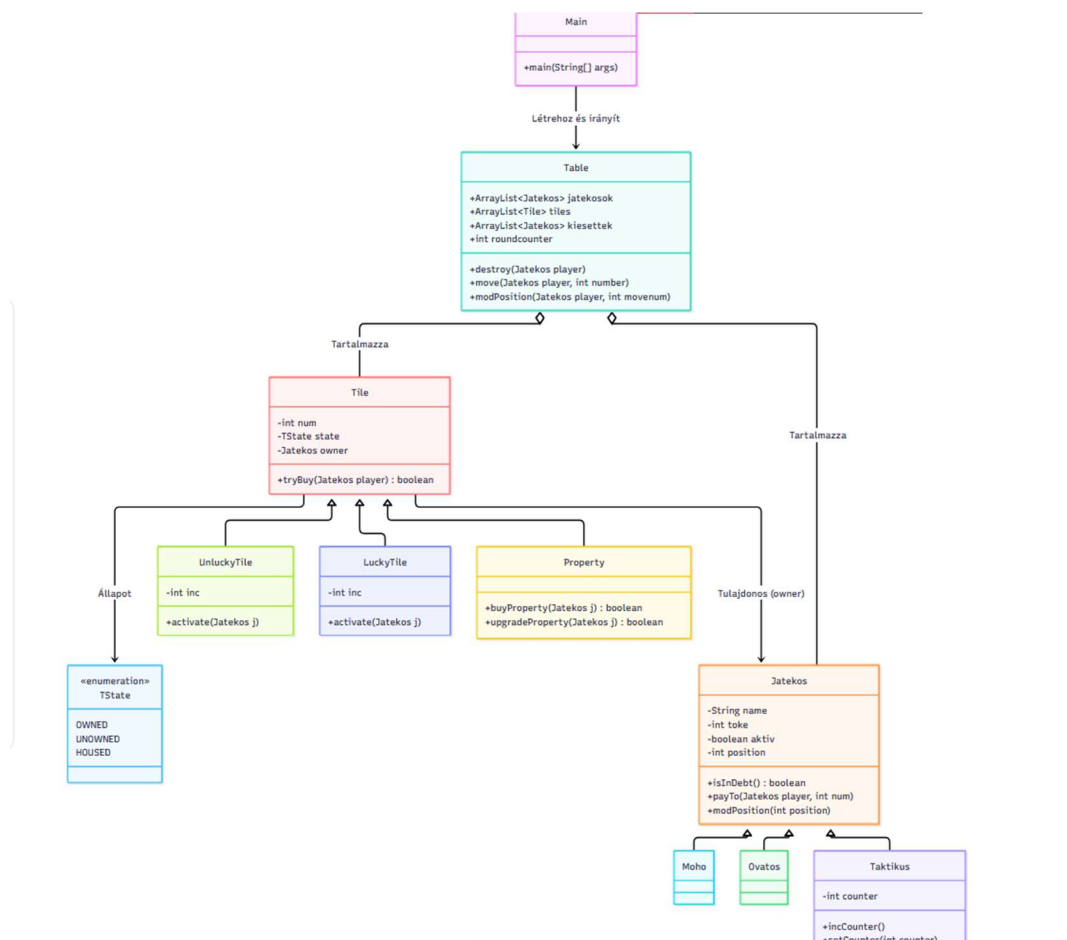
Ha a bekérés megvan, létrehoz egy „table” objektumot, amiben el vannak tárolva az objektumaink. Majd soronkénti beolvasással, és végrehajtással elkezdődhet a játék. A table.move() metódus oldja meg a játékosok mozgását, amiben „mozgatja” a játékost (mod-os osztással garantálva, hogy nem fut le a pályáról), majd kiválaszt egy landedOn mezőt, aminek típusát megállapítva levon/hozzáad pénzt a játékos vagyonához, vagy megpróbálja megvenni. Itt jön a képbe a visitor minta, a tryBuy függvény egy játékost kér be, és ha a feltételek adottak, meghívja a buyProperty() metódust, melyben a játékos megveszi, azaz levonja a pénzt, és tulajnak állítja be a játékost. A játékos stílusát a move függvény

vizsgálja, a Mohó stílus egyszerűen meghívja a tryBuy-t, a Taktikus a beépített számlálójával csak minden második körben, és az Óvatos-ra van egy feléltelvizsgálat is.

Mozgatás után a main ellenőrzi az isInDebt() függvénnyel, hogy kiesett-e az adott játékos, ha igen, akkor meghívja a destroy() függvényt, ami inaktívvá teszi az adott játékost, a kiesettek közé teszi, és lenullázza a telkeit.

Minden sor után ellenőrzi, hogy kiesett-e már kettő (vagy több) játékos, ha igen, akkor a kiesettek táblából kieszedi, és kiírja.

Osztálydiagram



Fontosabb metódusok

`tryBuy(Jatekos player)`: ellenőrzi, hogy a feltételek adottak-e, ha igen, meghívja a `buyProperty`, illetve `upgradeProperty` metódusokat

`move(Jatekos player)`: az egész lelke, ez felel a mozgatásért, és a vásárlások és tőkeváltoztatások indításáért

`modPosition(int position)`: ez felel a „tényleges” lépésért, innen tudjuk, hova landol egy játékos

Program futás közben

```
"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:M:\IntelliJ IDEA 2026.2.2\lib\idea  
Második kiesett: Zsofia, 2 kör alatt.
```

```
Process finished with exit code 0
```

```
"C:\Program Files\Java\jdk-25.0.2\bin\java.exe" "-javaagent:M:\IntelliJ IDEA 2026.2.2\lib\idea_rt.jar=65504" -Df  
File error.
```

```
java.io.FileNotFoundException: inputnincs.txt (A rendszer nem találja a megadott fájlt)  
    at java.base/java.io.FileInputStream.open0(Native Method)  
    at java.base/java.io.FileInputStream.open(FileInputStream.java:185)  
    at java.base/java.io.FileInputStream.<init>(FileInputStream.java:139)  
    at java.base/java.util.Scanner.<init>(Scanner.java:675)  
    at Main.main(Main.java:14)
```

```
Process finished with exit code 0
```

Felhasználói történések

AS A felhasználó

I WANT TO leszimulálni egy egyszerűsített Capitaly társasjátékot különböző stratégiájú játékosokkal, és megtudni, melyikük esik ki másodszorra a játékból.

1.

GIVEN	megadjuk futáskor az érvényes adatokat tartalmazó input.txt fájlt
WHEN	betöltjük a fájlban található pályát, játékosokat és a dobássorozatot
THEN	a program lefuttatja a köröket, és amint a második játékos tőkéje 0 alá csökken, kiírja a nevét a konzolra és kilép

2.

GIVEN	megadjuk futáskor a fájl nevét, input.txt
WHEN	a fájl nem létezik a mappában, vagy hibás az elérési útvonal
THEN	a program elkapja a kivételt, kiírja, hogy "File error." (vagy "File not found."), és hibaüzenet nélkül leáll

3.

GIVEN	megadunk egy bemeneti fájlt, amelyben a kezdőmező után rögtön nagy összegű (-6000) balszerencse mezők következnek
WHEN	a játékosok az első körökben rendre rálépnek ezekre a mezőkre
THEN	a program azonnal, a megadott dobások végigérése előtt azonosítja az első két csődbe ment játékost, és kiírja a második kiesett nevét

4.

GIVEN	megadunk egy fájlt, amelyben Mohó és Taktikus játékosok is szerepelnek, sok eladó ingatlannal
WHEN	a játékosok folyamatosan ingatlanra lépnek

THEN	megkapjuk az eredményt, amely helyesen tükrözi, hogy a Mohó játékos minden adandó alkalommal költ és hamarabb esik ki, míg a Taktikus minden második vásárlást kihagyja, így tovább életben marad
-------------	---

5.

GIVEN	megadunk egy fájlt, ahol egy játékos több ingatlant is birtokol, de egy balszerencse mező miatt csődbe megy
WHEN	a kiesése után egy másik, aktív játékos rálép a korábban általa birtokolt ingatlanra
THEN	a program nem von le bérleti díjat a rálépő játékostól, hanem engedélyezi számára a felszabadult gazdátlan telek megvásárlását

6.

GIVEN	megadunk egy érvényes fájlt, de a megadott dobások (körök) száma túl kevés ahhoz, hogy két játékos is csődbe menjen
WHEN	a program végigér a fájlban lévő összes dobáson
THEN	a program hiba és összeomlás nélkül lefut, a Table objektum rögzíti a végső állapotot, de mivel nincs meg a két kieső, nem ír ki eredményt

7.

GIVEN	megadunk egy fájlt, ahol egy Óvatos játékos lép rá egy gazdátlan ingatlanra
WHEN	a játékos tőkéje nem éri el a 2000 Petákot (a fele nem fedezi az 1000 Petákos vételárat)
THEN	az Óvatos játékos nem vásárolja meg az ingatlant, a mező állapota UNOWNED marad, és a játékos tőkéje nem csökken